AI Engineering Internship Case Study

Technical Assessment & Practical Implementation Task

INTERN CANDIDATE	ASSIGNED DATE	SUBMISSION DEADLINE	PRIMARY FRAMEWORKS
Ahmar	**27-07-2026**	**29-07-2026 (23:59 PKT)**	**LangGraph & LangChain**

1. Objective & Assessment Goal

The purpose of this case study is to evaluate your practical problem-solving ability, code structure, and architectural comprehension of state-of-the-art LLM orchestration using **LangGraph** and **LangChain**, combined with core **Python software engineering principles** (OOP, DSA, File I/O, and Robust Error Handling).

You are tasked with designing and implementing a runnable, end-to-end **Autonomous Technical Support & QA Agent System ("TechAssist-AI")** that ingests technical documentation, retrieves relevant knowledge, routes user queries based on context, executes parallel document sanity checks, and handles errors gracefully without crashing the graph.

2. Targeted Skill Matrix & Core Concepts

Your implementation must explicitly demonstrate mastery over the following technical building blocks:

1. LangGraph Execution Flow Patterns
Sequential nodes, Parallel branch processing, Conditional routing (edges), and Iterative loops/cycles (e.g., reflection/retry).

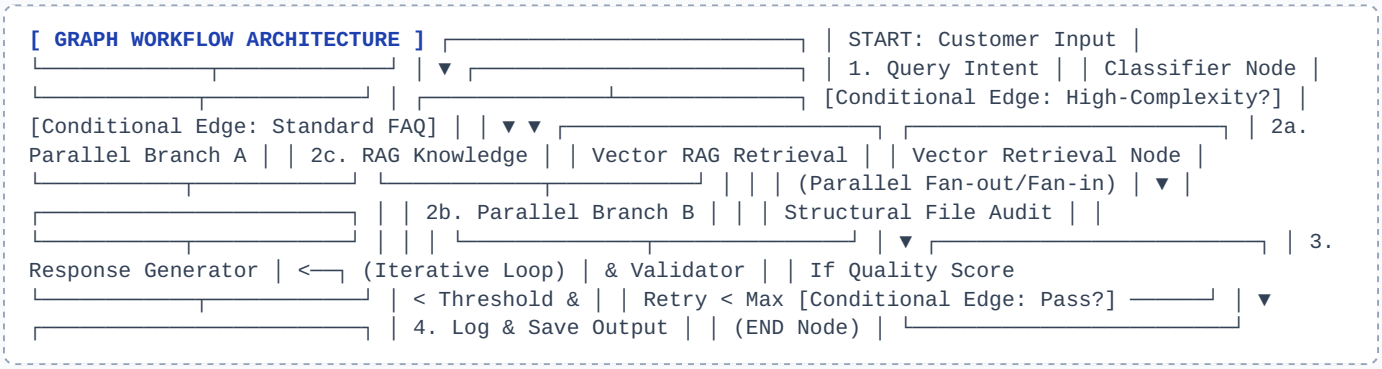
2. Basic RAG Architecture
Document loading, text chunking/splitting, vector embedding indexing, and similarity search retriever setup.

3. LangChain Components
PromptTemplates, Chat Models / LLM wrappers, Output Parsers, Tool execution, and custom State schemas.

4. Core Python Proficiency (Target: 7/10)
Clean OOP, structured File Handling (JSON/Markdown), efficient DSA (dict/hash map lookups, queue/stack state), and robust Try-Except Error Handling.

3. Problem Statement: "TechAssist-AI Workflow"

A software company maintains local technical documentation files (e.g., `docs/api_guide.md`, `docs/troubleshooting.txt`, and `docs/config.json`). Customer support receives multi-faceted queries ranging from general technical FAQs to complex troubleshooting requests requiring automated validation.



4. Detailed Implementation Requirements

Part A: Python Engineering & Document Ingestion (OOP + File Handling + DSA)

- **Class Architecture (OOP):** Create a class `DocumentManager` responsible for reading, verifying, and indexing documentation files.
- **File Handling:** Implement methods to parse `.txt`, `.md`, or `.json` knowledge base files from a designated directory. Handle non-existent paths and corrupted files cleanly.
- **DSA Optimization:** Maintain a fast cache mechanism using Python dictionaries (hash maps) or sets to store pre-calculated document hashes or keyword indices for quick lookup before triggering LLM calls.

Part B: LangChain & RAG Pipeline

- **Text Chunking & Embeddings:** Use LangChain's `RecursiveCharacterTextSplitter` (or equivalent) to chunk document text.
- **Vector Indexing:** Store vectors using an in-memory vector store (e.g., FAISS, Chroma, or `InMemoryVectorStore`).
- **Retrieval Chain:** Build a custom retrieval chain with `ChatPromptTemplate` that accepts user query context and returns sourced responses.

Part C: LangGraph Construction (The Core Task)

Construct a `StateGraph` with a defined `TypedDict` state (e.g., `AgentState` storing `query`, `category`, `retrieved_docs`, `generated_response`, `retry_count`, `error_logs`):

1. **Sequential Concept:** Connect initial entry nodes sequentially (e.g., `IntentClassifier` → `Retriever`).
2. **Conditional Concepts:** Add conditional edges using a routing function to decide whether a query requires RAG retrieval, structural document file checks, or a direct response.
3. **Parallel Execution Concept:** For technical bug/troubleshooting queries, trigger two nodes concurrently in parallel (e.g., Node A: Vector Retrieval vs. Node B: File System Log/Config Audit) and merge their results prior to answer generation.
4. **Iterational/Loop Concept:** Implement a loop edge where the generator node assesses its own answer clarity/relevance score. If the score is below a threshold (e.g., < 0.7), increment `retry_count` and loop back to regenerate/refine the answer (max 2 retries).

Part D: Exception & Error Handling Strategy

- Ensure **every node function** is wrapped in robust `try-except` blocks.
- If the vector store fails or returns zero results, the graph must handle the exception gracefully, log the failure into `state['error_logs']`, and reroute to a fallback static help node without throwing an unhandled runtime error.
- Save execution history and final output to a clean local JSON file (`execution_log.json`).

5. Recommended State Schema Reference

```
from typing import TypedDict, List, Dict, Any, Optional

class AgentState(TypedDict):
    user_query: str
    category: str  # "FAQ", "Troubleshooting", "System_Audit"
    retrieved_docs: List[str]
    file_audit_results: Dict[str, Any]
    generated_response: str
    quality_score: float
    retry_count: int
    error_logs: List[str]
    is_complete: bool
```

6. Deliverables & Submission Checklist

Deliverable	Format / Filename	Description
1. Code Base	`main.py` or Jupyter Notebook	Fully executable Python script/notebook containing LangGraph workflow, LangChain components, and custom classes.
2. Knowledge Base Data	`/docs` folder	At least 2-3 sample markdown/text files containing mock technical documentation.
3. Execution Log Output	`execution_log.json`	Automatically generated output showing state transitions, error logs, and final output for at least 2 test queries.
4. Brief Case Study Report	`README.md`	A short explanation (1-2 pages) describing how parallel, conditional, and iterative nodes were implemented.

7. Evaluation Rubric

Evaluation Criteria	Weightage	Key Focus Area
LangGraph Mastery & Workflow Design	35%	Correct implementation of parallel nodes, conditional routing edges, and iterative retry loops.
Python Code Quality & OOP	25%	Clean code layout, modular design, typing, and efficient data structures (DSA & File I/O).
RAG & LangChain Integration	20%	Proper document splitting, embeddings, vector storage retrieval, and prompt template assembly.
Error Resilience & Exception Handling	15%	Graceful exception handling in nodes, logging errors in state, fallback mechanisms.
Documentation & Completeness	5%	Clear README file, structured deliverables, and meeting deadline.

> **Submission Note:** Submit your GitHub repository link or compressed zip folder to your supervisor by **29-07-2026 at 23:59 PKT**. If you run into API rate limits or mock environment setups, stub the LLM responses cleanly using LangChain's standard mock wrappers or deterministic test outputs.